LibreAssistant: Technical Specification and Architectural Framework

Introduction

Purpose

This document serves as the canonical technical specification for the LibreAssistant project. It translates the philosophical, ethical, and design principles articulated in the foundational research paper, "Vitruvian AI: LibreAssistant Design Framework," into a concrete engineering blueprint. The objective is to provide a comprehensive and actionable guide for core developers, community contributors, and project stakeholders, ensuring that the final system is a faithful and robust implementation of its guiding philosophy.

Guiding Principles

The architecture detailed herein is governed by the Vitruvian triad of *firmitas* (durability, robustness), *utilitas* (utility, function), and *venustas* (beauty, harmony), as interpreted for a modern, open-source AI system. Every technical decision, from the choice of a programming language to the design of a user interface element, is measured against these three interdependent ideals. *Firmitas* is achieved through a secure, maintainable, and resilient architecture. *Utilitas* is realized in a system that is genuinely useful, respects user autonomy, and serves a constructive purpose. *Venustas* is manifested in a user experience that is intuitive, harmonious, and aesthetically pleasing, recognizing that beauty is integral to building user trust and enhancing usability.

Scope

This specification provides an exhaustive definition of the LibreAssistant project. It covers the project's foundational governance, ethical commitments, legal framework, technology stack, system and data architecture, user experience philosophy, core feature set, development and deployment operations, and licensing. This document is intended to be a living blueprint, version-controlled and updated in tandem with the project's evolution, yet always anchored to the foundational principles established in the Vitruvian framework.

Part I: Foundational Principles & Governance (The Architect's Education)

Core Principle

This section translates the principles of Book I of *De architectura*, which focuses on the "holistic education of the architect". For LibreAssistant, this concept is embodied in the project's foundational governance, ethical commitments, and legal framework. These elements serve to

educate and guide all contributors, ensuring that every participant, regardless of their specialization, understands the full scope and responsibility of their work. A successful system requires architects—its developers and leaders—who possess a competency extending far beyond mere technical proficiency into the realms of philosophy, ethics, law, and design.

Specification 1.1: The "Architect's Oath" (Project Constitution)

The project will establish and maintain a public, version-controlled document titled "The Architect's Oath". This document will serve as the project's constitution, a supreme charter that articulates its core values and non-negotiable commitments.
The Oath will explicitly state the project's unwavering dedication to:

- The Four Essential Freedoms of Free and Open-Source Software (FOSS), guaranteeing user liberty.
- The principles of ethical, human-centered AI, specifically Fairness, Accountability, and Transparency (FAccT), to ensure the system acts in the user's best interest.
- Unyielding user privacy and data autonomy, architected through a "privacy by design" methodology.
- The practice of holistic, collaborative, and human-centered design, which prioritizes user well-being and avoids manipulative patterns.

This document will function as the primary guiding star for all architectural decisions, feature development, and community moderation. It is both a transparent promise to its users and a binding charter for all contributors. The Oath functions as a formal social contract and a primary risk management tool. By codifying non-negotiable ethical boundaries from the outset, it provides a clear, objective framework for decision-making and conflict resolution. This process transforms the Oath from a mere mission statement into an active governance mechanism. Any proposed feature or contribution can be measured against its tenets, for instance: "Does this action uphold the user's right to data portability as defined by our GDPR commitment?" This structured evaluation mitigates the risk of "ethical drift" by making core principles a non-negotiable part of the development and review process, thereby ensuring the project's structural and philosophical integrity—its *firmitas*—over time.

Specification 1.2: Core Philosophy (The Four FOSS Freedoms)

The LibreAssistant project officially adopts the Free Software Foundation's "Four Essential Freedoms" as its defining philosophical core. This commitment is absolute and informs all subsequent decisions regarding licensing, technology selection, and contribution policies. The name "LibreAssistant" itself is a declaration of this principle, emphasizing freedom (*libre*) over price.
The Four Essential Freedoms are defined as follows :

- **Freedom 0:** The freedom to run the program as you wish, for any purpose.
- **Freedom 1:** The freedom to study how the program works, and change it so it does your computing as you wish. Access to the source code is a precondition for this.
- **Freedom 2:** The freedom to redistribute copies so you can help others.
- **Freedom 3:** The freedom to distribute copies of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes. Access to the source code is a precondition for this.

The implication of this mandate is that LibreAssistant must always empower its users, granting them ultimate control over the software they run. This inherently prevents vendor lock-in and

ensures that the assistant serves the user, not a corporate entity. This philosophy must extend to commercial use, development, and distribution, as these are vital for the health and success of the FOSS ecosystem.

Specification 1.3: Ethical Framework (FAccT Principles)

All AI components, algorithms, and data handling practices within LibreAssistant must strictly adhere to the principles of Fairness, Accountability, and Transparency (FAccT). This framework is essential for building a trustworthy AI that mitigates harm and acts in the user's best interest.

- **Fairness:** The project must implement proactive and continuous measures to identify, measure, and mitigate biases in datasets and models. This includes performing audits for demographic and subgroup biases and designing the system to avoid creating or reinforcing inequitable social outcomes. The goal is not an unachievable "neutrality" but a demonstrable commitment to justice and equity.
- **Accountability:** The project, its core maintainers, and its governance structure are accountable for the system's operation and its impact. Clear mechanisms for users to report issues, seek redress, and understand decisions must be established. Features such as the "Bill of Materials" dashboard (Part II) and the "System Health" dashboard (Part IX) are key technical implementations designed to enable this accountability by making the system's composition and behavior observable.
- **Transparency:** The system's logic, data sources, and decision-making processes must be as transparent as possible to the user. This principle extends beyond merely having open-source code. It requires a commitment to explainable AI (XAI) techniques where appropriate and, more fundamentally, a UI/UX design that clearly communicates the "why" behind any information presented or action taken by the assistant. The user should never be left wondering why the assistant provided a certain answer.

Specification 1.4: Legal & Data Governance Mandate (GDPR & Privacy by Design)

The LibreAssistant project must be architected from the ground up to comply with the most stringent data privacy regulations, using the European Union's General Data Protection Regulation (GDPR) as the baseline standard for all users, regardless of their location. This commitment to "Law" is a non-negotiable aspect of the architect's education.
The project will implement the core GDPR principles for all personal data processing :

1. **Lawfulness, Fairness, and Transparency:** Processing must have a lawful basis, be fair, and be transparent to the user.
2. **Purpose Limitation:** Data must be collected only for specified, explicit, and legitimate purposes.
3. **Data Minimization:** Only the minimum amount of data necessary for the specified purpose shall be collected.
4. **Accuracy:** Personal data must be accurate and kept up to date.
5. **Storage Limitation:** Data must not be kept in an identifiable form for longer than is necessary.
6. **Integrity and Confidentiality:** Data must be processed in a manner that ensures appropriate security.
7. **Accountability:** The project must be able to demonstrate compliance with these

principles.

To achieve this, "Privacy by Design" and "Privacy by Default" are mandatory development methodologies. Every new feature must be evaluated for its privacy implications before development begins, and the default settings for any feature must be the most privacy-preserving option.

Specification 1.5: Community Structure (Cross-Disciplinary "Guilds")

To embody the Vitruvian ideal of the polymath architect and prevent the kind of siloed thinking that leads to architectural failures, the project's community will be structured to foster interdisciplinary collaboration. This structure will be modeled on a combination of medieval guilds and modern DevOps practices, establishing cross-functional "Guilds."

Examples of these guilds include:

- **The Ethics Guild:** Responsible for reviewing new features and architectural changes for compliance with the FAccT framework and the "Architect's Oath."
- **The UX Guild:** Responsible for maintaining and evolving "The Libre Order" design system (Part IV).
- **The Security Guild:** Responsible for proactive security audits, threat modeling, and incident response planning.
- **The Data Governance Guild:** Responsible for overseeing the project's adherence to data sourcing policies, FAIR principles, and GDPR compliance.

These guilds will not be gatekeepers but collaborative bodies, bringing together developers, designers, legal experts, ethicists, and end-users to provide holistic review and guidance throughout the development lifecycle. This structure ensures that the full spectrum of Vitruvian knowledge is applied continuously, safeguarding the project's long-term health and integrity.

Part II: Technology Stack & Core Components (Building Materials)

Core Principle

This section translates the principles of Book II of *De architectura*, which focuses on the properties and appropriate use of "Building Materials". In modern software, the technology stack is the equivalent of stone, timber, and concrete. The choice of these "materials"—the programming languages, frameworks, and databases—has profound and lasting consequences for the project's durability (*firmitas*), performance, and maintainability. For LibreAssistant, these materials must be chosen to reflect the core philosophy of freedom, transparency, and community ownership.

Specification 2.1: A Pure FOSS Technology Stack

The project's core technology stack must consist exclusively of components governed by recognized Free and Open-Source Software licenses. This is a foundational, non-negotiable requirement that aligns with the "libre" ethos of the project. It ensures freedom from vendor lock-in, allows for maximum transparency and auditability, and guarantees that the project's foundational "materials" grant the same four freedoms that the project itself provides to its users.

The rationale for the selection of each component must be publicly documented, much as Vitruvius explained his criteria for choosing different types of timber, providing clarity for all contributors.

Table 1: LibreAssistant Core Technology Stack

The following table specifies the approved "building materials" for the project's core services. This serves as a single source of truth to ensure consistency and prevent the introduction of non-compliant proprietary components. This formal declaration operationalizes the principle that the rationale for material selection must be transparent and documented.

Component Category	Technology	License Type	Rationale
Programming Language	Python	PSF License	The de facto standard for AI/ML development, offering an extensive ecosystem of libraries, robust community support, and a permissive, business-friendly license.
AI/ML Frameworks	PyTorch, TensorFlow	BSD-style, Apache 2.0	Industry-standard frameworks providing powerful, well-documented, and highly optimized tools for model development and deployment under permissive licenses.
Database	PostgreSQL	PostgreSQL License	A powerful, highly reliable, and feature-rich object-relational database system known for its standards compliance and a permissive, MIT-style license.
Containerization	Docker, Podman	Apache 2.0	The industry standard for creating lightweight, portable, and reproducible application environments, essential for a distributed development community and scalable deployment.
Web Framework	FastAPI / Flask	MIT / BSD	Modern,

Component Category	Technology	License Type	Rationale
			high-performance Python frameworks ideal for building the efficient and robust APIs required for the assistant's modular architecture.
Version Control	Git	GPLv2	The universal standard for distributed version control, enabling asynchronous and collaborative development within the global open-source community.

Specification 2.2: Ethical Data Sourcing & FAIR Principles

The most critical raw material for any AI system is data. Therefore, all data used for training LibreAssistant models must be ethically sourced and managed according to the FAIR Data Principles (Findable, Accessible, Interoperable, Reusable). An AI built on ethically opaque data is built on a flawed foundation, inherently lacking the integrity required for a trustworthy system.

- **Ethical Sourcing:** All training data must be derived from publicly available, permissively licensed, or explicitly user-consented datasets. The use of scraped data of ambiguous origin or restrictive licensing is strictly prohibited.
- **Data Provenance:** The origin, licensing, and complete transformation history of every dataset must be meticulously tracked and documented. This documented trail is essential for debugging models, auditing for bias, and providing users with transparency about the information shaping the assistant's intelligence.
- **FAIR Implementation:** To ensure data is a high-quality, reusable asset for the community, the project will adhere to the FAIR principles :
    - **Findable:** All approved datasets will be described with rich metadata and indexed in a searchable public catalog, each assigned a globally unique and persistent identifier (e.g., a DOI).
    - **Accessible:** The metadata will be retrievable by its identifier via a standardized, open protocol. The metadata will remain accessible even if the data itself is retired.
    - **Interoperable:** Data and metadata will use formal, shared, and broadly applicable languages for knowledge representation, such as standardized vocabularies and ontologies, to allow for machine processing.
    - **Reusable:** All datasets will be released with a clear and accessible data usage license (e.g., Creative Commons) and will be associated with detailed provenance information.

Specification 2.3: Feature - The "Bill of Materials" Dashboard

To embody the principle of transparent components, LibreAssistant will include a user-facing dashboard that provides a complete, interactive "Bill of Materials". This feature moves beyond

the standard open-source practice of simply making code visible and pioneers a new standard of holistic system auditability. This dashboard applies the FAIR data principles not just to external training data but to the software system itself, treating the system's own composition as a dataset to be made Findable, Accessible, Interoperable, and Reusable. By making dependencies and data origins clear and traceable, this feature directly enhances the system's durability (*firmitas*) and builds profound user trust (*venustas*).
The dashboard will provide a clear, human-readable interface allowing any user to inspect:

- **Software Components:** A comprehensive list of all FOSS libraries and dependencies used in the project, including their exact versions and direct links to their respective licenses and source repositories.
- **AI Models:** The core AI models currently being utilized by the assistant, with links to their source code, the research papers that describe their architecture, and any known operational limitations.
- **Training Datasets:** A complete catalog of the datasets used for training the core models, detailing their origins, licensing terms, and a transparent summary of their characteristics, including any known or potential biases identified during audits.

Part III: User Experience & Human-Centered Design (On Temples)

Core Principle

This section translates the principles of Book III of *De architectura*, which establishes the human body as the ultimate canon of proportion for the design of temples. In modern software, the "sacred space" of the temple is the User Experience (UX). The user is the modern "Vitruvian Man"—the standard against which all design decisions are measured. A well-designed UX, like a well-proportioned temple, feels intuitive, harmonious, and deeply satisfying, fostering a sense of trust and competence in the user. Therefore, every interaction must be crafted according to principles of proportion, clarity, and human-centric harmony.

Specification 3.1: Principle of Harmonious Interaction

The design of LibreAssistant must be fundamentally human-centered. Its capabilities, interface, and mode of interaction must be scaled to human cognitive abilities and emotional needs, not the other way around. The design must be aesthetically pleasing, recognizing the well-documented **aesthetic-usability effect**, which shows that users perceive beautiful designs as more usable and trustworthy. This confirms the Vitruvian insight that beauty (*venustas*) is not a decorative luxury but is integral to function (*utilitas*). A beautiful interface builds user confidence and makes the system more pleasant to engage with, directly enhancing its utility.

Specification 3.2: Transparent and Symmetrical UI/UX

- **Transparent UI:** The interface must be architected to be transparent about its reasoning. When the assistant presents information or performs an action, the UI must provide clear affordances for the user to understand *why*. This includes citing its sources, explaining its logic in a clear and hierarchical manner, and allowing the user to verify the information.

This is the modern equivalent of revealing the clear, rational geometry of a temple's design.

- **Ethical Symmetry:** The design must treat all user choices with absolute symmetry. This is an ethical application of the principle of balance and a direct rejection of manipulative "dark patterns". It must be as easy, in terms of clicks and cognitive load, for a user to decline a permission, opt-out of data sharing, or reverse a setting as it is to opt-in. This design mandate is more than a UX guideline; it is a direct, technical implementation of the project's core FOSS philosophy. The FOSS philosophy is fundamentally about granting users freedom and control. Dark patterns are, by definition, interfaces designed to subvert that freedom for the benefit of the system owner. By mandating "Ethical Symmetry" in the technical specification, the project builds a technical guardrail that enforces its political commitment to user freedom at the most granular level of interaction, ensuring the system's *utilitas* is never achieved at the expense of the user's liberty.

Specification 3.3: Feature - The "Vitruvian" UI Framework

To ensure consistency, harmony, and quality across the entire user experience, especially within a modular ecosystem, LibreAssistant will be built upon a comprehensive, open, and well-documented UI framework. This framework serves as the project's own "classical order," providing a standardized and reusable toolkit for all user-facing elements.
The framework will consist of the following core components :

- **A Proportional Typographic Scale:** A mathematically defined and harmonious scale for all text elements (headings, subheadings, body text, captions) will be established to ensure a clear and consistent visual hierarchy across the application.
- **A Responsive Modular Grid:** A flexible, powerful grid system will underpin all layouts. This ensures that interfaces remain balanced, ordered, and harmonious across all screen sizes and devices, translating the static symmetry of classical architecture into the dynamic context of modern digital products.
- **A Consistent Component Library:** All interactive elements—such as buttons, sliders, input fields, and menus—must be standardized in their visual style, interactive behavior, and accessibility implementation. This creates a predictable, coherent, and trustworthy user experience.

Specification 3.4: Feature - The Onboarding Experience

This specification details the step-by-step user flow for onboarding, designed to introduce core concepts of modularity and privacy from the very first interaction.
Step 0 — Welcome Screen

- **Visual:** LibreAssistant logo, clean, minimal interface.
- **Tagline:** “Open. Tolerant. Yours.”
- **Action:** Get Started button.

Step 1 — Engine vs Car Metaphor

- **Visual:** Car body labeled LibreAssistant with swappable “engine blocks” representing AI provider logos.
- **Copy:** “LibreAssistant is your car — the AI model you choose is its engine. Switching engines changes performance, not the car itself.”
- **Action:** Choose Engine Type.

Step 2 — Cloud vs Local Choice Two large selectable cards:

1. **Cloud Engine**
    - **Subtext:** “Run AI in the cloud with your API keys. Works anywhere.”
2. **Local Engine**
    - **Subtext:** “Run AI on your device for maximum privacy.”

Step 3 — Provider Selection (Same list for Cloud & Local) Seven Available Providers:

1. OpenAI
2. Google
3. Meta
4. xAi
5. DeepSeek
6. Anthropic
7. HuggingFace

- **If Cloud:** API key entry or OAuth login (if supported).
- **If Local:** Pre-tested quantized model downloads for that provider (where available). Option to load a custom local model.

Step 4 — Plugin & Switchboard Setup

- **Visual:** Empty switchboard grid with “+” slots.
- **Copy:** “LibreAssistant uses plugins to help with tasks. Add the ones you need now — you can change them anytime.”
- **Actions:**
    - Enable recommended plugins.
    - Browse optional plugins.

Step 5 — Privacy & Transparency

- **Visual:** Glass box + lock icons.
- **Copy:** “LibreAssistant runs in a glass box. You see exactly which plugins are active, and your data stays under your control.”

Step 6 — Done Screen

- **Visual:** Preview of the Switchboard tab.
- **Copy:** “All set! Enter a request in the switchboard to get started. Plugins will light up as they’re used.”
- **Action:** Go to Switchboard.

Part IV: System Architecture & Design System (The Orders of Architecture)

Core Principle

This section translates the principles of Book IV of *De architectura*, which details the distinct characteristics and proportional rules of the architectural orders (Doric, Ionic, Corinthian). Each order has a specific character and is appropriate (*decor*) for a particular purpose. In software, this finds a direct parallel in high-level **software architectural patterns** and **design systems**. The choice of pattern is a critical decision of appropriateness for the task at hand, and the design system provides the coherent, systematic language for its expression.

Specification 4.1: Architectural Pattern - Modular Microkernel

After careful consideration of appropriateness (*decor*), LibreAssistant will be architected using a **modular or microkernel architecture**. This pattern is the most fitting "order" for a flexible, community-driven, and extensible AI assistant.

- **Structure:** The system will be composed of a stable, minimal central "core" (the microkernel). This core is responsible for essential services: managing the main application loop, handling user state, processing core inputs, and exposing a stable API for modules. All other functionalities, or "skills" (e.g., weather, calendar, civic info), will be implemented as independent "plug-in modules" that communicate with the core exclusively through this well-defined API.
- **Rationale:** This architectural choice provides the ideal balance of stability and flexibility. The core can be rigorously tested and maintained for maximum durability (*firmitas*), while the modular structure allows the open-source community to contribute new capabilities and skills without risking the destabilization of the entire system. This technical choice is also a symbolic expression of the project's philosophy: it is an open invitation for collaboration and extension.

Specification 4.2: Feature - "The Libre Order" Design System

To ensure a coherent and high-quality user experience across all modules contributed by a diverse community, the project will be governed by **"The Libre Order,"** a comprehensive and publicly accessible design system. This system is the modern analogue to the Renaissance treatises that codified the classical orders, providing a shared language and a standardized toolkit for all contributors.

- **Description:** "The Libre Order" will be a living, version-controlled project in its own right, consisting of a public repository and an accompanying documentation website. It will be developed using open-source UI/UX design tools like Penpot or Pencil Project to ensure it remains fully aligned with the project's FOSS ethos.
- **Contents:** The design system will codify the assistant's entire user-facing presence and will contain :
    - **Design Principles:** The foundational philosophy of the UI/UX, emphasizing transparency, accessibility, and the "Principle of Harmonious Interaction."
    - **Component Library:** A complete library of reusable UI components (buttons, cards, modals, etc.) with ready-to-use code snippets and detailed usage guidelines.
    - **Content Guidelines (Voice and Tone):** A comprehensive style guide defining the assistant's personality—its "character"—including its tone of voice, vocabulary, and interaction patterns to ensure a consistent and trustworthy persona.
    - **Visual Language:** The official color palette, typography scale, iconography set, and spacing rules that establish a recognizable and coherent visual identity.

The combination of a microkernel architecture and an open design system creates a powerful dynamic of "governed flexibility." The architecture provides a stable foundation (*firmitas*), while the design system provides a coherent aesthetic and interaction language (*venustas*). This allows a decentralized community to build a diverse yet unified product (*utilitas*). The microkernel's API defines the *functional* contract for any new module, while the design system's components define the *experiential* contract. This two-part contract enables the project to scale its community contributions without sacrificing quality or coherence, solving the classic open-source challenge of creating a unified product from many disparate parts.

Table 2: "The Libre Order" Core Component Specification

The following table provides a quick-reference guide for the specification of core components within the design system. This ensures that all new interfaces and skills developed by the community feel native to the assistant, creating the visual and interactive harmony (*eurythmia*) that is essential for user trust.

Component	State	Accessibility (WCAG 2.1)	Description
Primary Button	Default, Hover, Active, Disabled	AA Contrast, Focus Visible	Used for the primary call to action on a page or within a module.
Input Field	Empty, Filled, Focus, Error	AA Contrast, Associated Label	Standard field for user text input, with clear state indicators.
Information Card	Default, Hover		A self-contained unit for displaying information, with a consistent structure for heading, body, and optional actions.
Modal Dialog	Active	Focus Trapped, ESC to close	An overlay dialog that requires user interaction, with strict focus management for accessibility.

Part V: Civic Function & Public Discourse (Public Buildings)

Core Principle

This section translates the principles of Book V of *De architectura*, which details the design of public buildings like forums, theaters, and basilicas. These structures are defined by their **civic function and public utility (*utilitas*)**. Their architecture actively shapes the nature of the community life and discourse within them. In the 21st century, much of public discourse has migrated to the "digital public sphere". As a tool for accessing and processing information, LibreAssistant has a civic responsibility to be architected not merely as a private utility but as a healthy, constructive gateway to this digital commons.

Specification 5.1: Principle of Civic Utility and Open Discourse

The design of LibreAssistant must actively promote a healthy information ecosystem and support informed public discourse. The architectural choices of today's dominant social media platforms, which prioritize engagement at all costs, have created "unhealthy" digital conditions that fuel misinformation and polarization. LibreAssistant must be architected as a direct counterpoint to this trend.

- **Information Veracity:** The assistant's architecture for sourcing information from the

internet must be designed to prioritize reliable, authoritative sources. It must incorporate mechanisms to identify and flag potential misinformation, not to censor, but to provide the user with context for critical evaluation. It must function as a tool for critical thinking, not as a passive amplifier for unverified content.

- **Combating Polarization:** The assistant must be architected to deliberately resist the creation of filter bubbles and echo chambers. When queried on a contentious topic, its core logic must not be to provide a single, algorithmically optimized "answer." Instead, its primary function must be to present multiple, well-sourced perspectives in a structured and neutral manner. This is a deliberate architectural choice to build a pluralistic digital forum, not a partisan echo chamber.

Specification 5.2: Feature - "Public Square" Mode

To operationalize its civic function, LibreAssistant will incorporate a specific mode or a suite of skills designed for civic engagement, transforming it into a powerful civic technology tool. This mode will include:

- **Civic Information Skill:** A feature that provides users with easy, structured access to information about their elected officials (at all levels of government), upcoming elections (including polling locations and ballot information), public government meetings, and public data portals. This skill will integrate directly with existing, reliable civic tech platforms and government APIs.
- **Debate Summarizer Skill:** A tool capable of ingesting a link to a political debate, a long-form news article, or a policy paper and generating a neutral, structured summary of the different arguments presented, including the key claims and evidence cited by each side. This empowers users to quickly and efficiently understand complex public issues from multiple viewpoints, fostering a more informed citizenry.

Specification 5.3: Community as a Public Space

The development community for LibreAssistant—its forums, chat channels, and code repositories—is itself a public space. Its "architecture" must be intentionally designed to foster healthy, inclusive, and productive discourse.

- **A Robust Code of Conduct:** A clear, comprehensive, and strictly enforced code of conduct is the foundational "law" of this digital space. It will be prominently displayed and must be agreed to by all participants, ensuring the environment remains respectful and welcoming to contributors from all backgrounds.
- **Effective Moderation:** The community platforms must be equipped with effective moderation tools and a dedicated, trained moderation team. The goal of moderation is not to stifle debate but to manage conflict constructively and prevent harassment, ensuring the space remains a forum for civil, not violent, discourse.

Part VI: User Privacy & Personalization (Private Dwellings)

Core Principle

This section translates the principles of Book VI of *De architectura*, which focuses on the design of private dwellings (*domus*). A Roman house was carefully designed to adapt to its climate and site, reflect the owner's status, and mediate the boundary between public and private life. In the digital realm, every user's collection of devices, accounts, and data constitutes their **"personal digital space."** LibreAssistant's architecture must treat this space with the utmost respect, acting as an empowering and trustworthy guest, not an intrusive owner. The design must be founded on the principles of user autonomy and contextual adaptation, skillfully balancing powerful personalization with absolute user control and uncompromising privacy and security.

Specification 6.1: Privacy-by-Design and Privacy-Preserving AI

All features must be engineered with a "privacy by design" and "privacy by default" methodology, in full alignment with the GDPR principles outlined in Part I. Data collection will be minimized to what is strictly necessary for a feature to function and will always require explicit, granular, opt-in consent.
To provide advanced functionality without compromising this core tenet, the project will prioritize and invest in research and implementation of privacy-preserving AI techniques:

- **On-Device Processing:** Whenever feasible, computation, inference, and data processing will occur on the user's local device, preventing sensitive data from ever leaving their control.
- **Federated Learning:** For features that require learning from user behavior, models will be trained across a decentralized network of devices. Only the aggregated, anonymized model updates are shared, not the raw user data itself.
- **Differential Privacy:** Where data aggregation is necessary, statistical noise will be mathematically injected into the dataset to make it impossible to re-identify any single individual, providing formal privacy guarantees.

Specification 6.2: Feature - The "Personal Data Vault"

To make the abstract principles of data privacy tangible and controllable for the user, LibreAssistant will manage all user-specific data within a secure, encrypted "Personal Data Vault". This vault will be a dedicated, clearly marked section within the application's settings, designed to be the user's single point of control for their personal digital space.
The Vault's architecture must provide the following functionalities, which directly implement the user's data rights under GDPR :

- **Transparency:** A clear, human-readable log of all personal data the assistant has stored about the user, including learned preferences, location history, common queries, and any other user-specific information.
- **Control:** Granular controls allowing the user to view, edit, and permanently delete any individual piece of data in the vault at any time.
- **Portability:** A one-click option to export the entire vault in a standard, machine-readable format (e.g., JSON), ensuring the user's data is never locked into the system and can be moved to another service.

Table 3: "Personal Data Vault" User Rights and GDPR Mapping

The following table explicitly connects the "Personal Data Vault" features to their legal and ethical mandate under GDPR. This serves as a compliance checklist for developers and a

transparency report for users and auditors, demonstrating how the software's architecture respects fundamental user rights. This direct mapping makes abstract legal principles concrete and actionable, building trust and ensuring accountability.

"Personal Data Vault" Feature	User Action	GDPR Right Fulfilled	Article
Data Log	View a log of all stored personal data.	Right of Access	Art. 15
Data Editor	Edit or correct personal information.	Right to Rectification	Art. 16
Delete Button	Permanently delete specific data points.	Right to Erasure ('to be forgotten')	Art. 17
Export Button	Download all data in a JSON format.	Right to Data Portability	Art. 20
Consent Manager	Granularly enable/disable data processing.	Consent	Art. 7

Specification 6.3: Graduated and Consensual Personalization

LibreAssistant will reject the common practice of making assumptions about users or personalizing their experience without explicit consent. Instead, it will employ a workflow that blends initial customization with ongoing, consensual personalization.

- **Initial Customization:** During the onboarding process, the assistant will explicitly ask the user to customize key preferences (e.g., "What are your primary interests?", "What news sources do you trust?", "What should I call you?").
- **Adaptive Learning with Consent:** Over time, the assistant may identify patterns in usage. Instead of acting on them unilaterally, it will make a suggestion and request consent. For example: "I've noticed you frequently ask for the weather in the morning. Would you like me to add a weather forecast to your daily briefing?" The personalization only becomes active after the user gives explicit, affirmative consent, perfectly balancing system intelligence with user control and autonomy.

Part VII: Finishes and Decoration (User Interface)

Core Principle

This section translates the principles of Book VII of *De architectura*, which focuses on the finishing details of a building: the stucco, paint, and decorative work that provide its final character and beauty (*venustas*). In software, these "finishes" correspond directly to the aesthetic layer of the User Interface (UI)—its visual design, theming, and skinning. This final surface is what the user directly interacts with, and its quality, craftsmanship, and material integrity profoundly shape their perception of the entire application. A beautiful "finish" builds trust and makes the underlying structure feel more robust and reliable.

Specification 7.1: Principle of Aesthetic Integrity and Customization

LibreAssistant's visual identity must be thoughtfully and professionally designed to be clean, accessible, and trustworthy. The default "out-of-the-box" theme should convey competence and

reliability, adhering to the principles of visual hierarchy, balance, and accessibility established in the design system.
Furthermore, as a personal tool that resides in the user's digital *domus*, the assistant must offer users deep and meaningful control over its appearance. The architecture must allow users to customize its "decoration" to match their own tastes, context, and identity. This is achieved through a robust theming engine.

Specification 7.2: Robust Theming Engine

The assistant's front-end architecture must be built with theming in mind from the ground up. This is not an afterthought but a core architectural requirement.

- **Design Tokens:** The entire UI will be styled using a system of design tokens—variables that represent foundational visual properties like colors, font sizes, and spacing units.
- **Engine Functionality:** The theming engine will allow for the easy swapping of token sets, enabling deep, consistent, and application-wide customization with a single action. This architecture ensures that themes are not just superficial color changes but can fundamentally alter the look and feel of the application in a coherent way.

Specification 7.3: Feature - The "Theme Marketplace"

To empower user customization and foster a vibrant community of designers and creators, LibreAssistant will incorporate a "Theme Marketplace" directly within its settings. This feature transforms the act of "decoration" from a static choice into a collaborative, open-source activity. The marketplace will feature:

- **First-Party Themes:** A curated selection of professionally designed themes created and maintained by the core team. This will include standard "Light" and "Dark" themes, a "High Contrast" theme for accessibility, and potentially other themes with distinct color palettes and personalities.
- **Community-Sourced Themes:** A platform where users can create, upload, and share their own themes with the wider community. Other users will be able to browse, install, and rate these community-created themes. The architecture must include a secure sandboxing mechanism to load these user-generated style sheets without creating security vulnerabilities.

The creation of the default theme and the theming engine itself will be an open process. The design process will be conducted publicly on platforms that allow community feedback on mockups and prototypes. The Theme Marketplace will be integrated with a public repository where designers can submit new themes via pull requests, subject to a review process for quality and security. This workflow embodies a culture of credited, collaborative creation, ensuring the project's aesthetic remains vibrant and user-driven.

Specification 7.4: Main User Interface Layout

This specification details the primary UI layout for both mobile and desktop platforms, including the structure and functionality of the main navigation tabs.
Tab Order The main navigation tabs will be presented in the following order:

1. Switchboard
2. Catalogue
3. Past Requests

4. User Profile

Platform-Specific Layouts

- **Mobile Layout:** A tab bar will be located at the bottom of the screen, with tabs arranged from left to right in the specified order.
- **Desktop Layout:** A tab bar will be located on the right side of the screen, with tabs arranged from top to bottom in the specified order.

A. Switchboard Tab

- An always-visible request input field is located at the top.
- A plugin grid with up to 12 slots that light up when active during a request.
- Users can click or tap a plugin slot to view its quick status, including its name, the data passed to it, and a preview of its output.
- A thin activity line above the grid shows the activation sequence of the plugins for the current request.

B. Catalogue Tab

- A searchable list of all available plugins.
- Single-plugin entries display an icon, name, description, and a function to copy a link to the plugin.
- Multi-plugin entries are collapsible to reveal their sub-plugins.
- Each plugin has a toggle to enable or disable it.

C. Past Requests Tab

- A list of previous interactions, showing the timestamp, the original request text, and the icons of the plugins that were used.
- An ellipses menu for each entry provides options to: “View output,” “View plugin usage log,” and “Re-run” the request.
- The plugin usage log shows detailed parameters and output summaries for each plugin involved in the request.

D. User Profile Tab

- Explicit identity fields for the user to define their name, role, and preferred tone for the assistant.
- Management sections for API keys for all seven supported providers (OpenAI, Google, Meta, xAi, DeepSeek, Anthropic, HuggingFace).
- Management sections for local models for all seven supported providers.
- Preferences for theme selection, accessibility settings, and defining default plugin sets.

Part VIII: Water (Data Flow and Integrity)

Core Principle

This section translates the principles of Book VIII of *De architectura*, which is dedicated to the science and engineering of water. Vitruvius considered water fundamental to life, and its proper management—finding, testing, and transporting it—essential for a city's health. In modern software architecture, the flow of water is a powerful metaphor for the **flow of data**. Data is the lifeblood of any modern application; it must be sourced, transported, processed, and delivered reliably and efficiently. The principles of Roman hydraulic engineering find direct parallels in the modern field of data engineering.

Specification 8.1: Principle of Data Integrity and Provenance

Every piece of data that LibreAssistant uses or presents to the user must be treated with the same rigor that a Roman engineer would apply to a city's water supply. The user's "health"—their ability to make sound judgments based on the information provided—depends on the quality of this data.

- **Reliable Data Pipelines:** The internal data pipelines that LibreAssistant uses to fetch, process, and cache information are the system's "aqueducts." They must be architected for reliability, observability, and resilience. As a FOSS project, the design of these pipelines must be public, allowing the community to inspect them for potential flaws, bottlenecks, or sources of bias.
- **Verifiable Provenance:** Just as Vitruvius stressed the importance of knowing the source of one's water, LibreAssistant must provide clear data provenance. When the assistant presents a piece of information (e.g., a factual answer, a news summary), it must be able to cite its source. The UI must make it easy for the user to trace the information back to its origin, allowing them to assess its credibility for themselves. This makes the principle of data provenance manifest in the user experience, building trust and empowering critical thinking.

Specification 8.2: Feature - "Data Source Quality Indicators"

To help users evaluate the "quality of the water" they are consuming, LibreAssistant's interface will include visual indicators for the sources of its information. This feature does not censor information but empowers the user with the necessary context to critically evaluate it, fulfilling the assistant's role as a tool for thought.
These indicators will include:

- **Source Badges:** Next to a piece of information, a small, clear icon or text badge will indicate the general type of source (e.g., "Peer-Reviewed Journal," "Established News Organization," "User-Generated Content," "Blog," "Government Publication").
- **Reliability Score:** For news sources, the assistant could integrate with established, independent third-party fact-checking and media bias rating organizations (e.g., Ad Fontes Media, NewsGuard) to display a simple, understandable reliability or bias score.

Specification 8.3: The "Data Governance Compact"

LibreAssistant's privacy policy will be framed not as a dense legal document but as a "Data Governance Compact" with the user. This document, written in plain, accessible language, makes data governance a transparent, bilateral agreement, building the user trust that is essential for a personal AI assistant.
The compact will explicitly state:

- **What data is collected** (the "springs").
- **How it is transported and stored** (the "aqueducts" and "cisterns"), including security measures.
- **How it is used to power the assistant's features** (the "fountains").
- **The user's absolute right to control, access, and delete their data** (turning off the "taps"), linking directly to the "Personal Data Vault."

Part IX: Astronomy and Timekeeping (System Monitoring)

Core Principle

This section translates the principles of Book IX of *De architectura*, which explores the relationship between architecture, astronomy, and the measurement of time. An architect, Vitruvius argued, must understand the cosmos to orient buildings and design clocks. This integration of a larger order into human construction finds its modern parallel in the concepts of **system monitoring, observability, and data synchronization**. Monitoring a complex software system is like being an astronomer: instead of stars, architects observe metrics, logs, and traces to understand the health and behavior of the system.

Specification 9.1: Principle of System Self-Awareness and Observability

LibreAssistant must be architected to be highly observable, not just by its developers, but also by its users. A lack of observability is a primary cause of catastrophic system outages; it is the digital equivalent of trying to navigate at sea without stars or a clock. The system needs to be able to monitor its own performance and health and communicate its state in a transparent way.

- **Internal Monitoring Stack:** The back-end infrastructure for LibreAssistant must include a robust, open-source monitoring stack. This will likely use Prometheus for metrics collection, Loki for log aggregation, and Grafana for visualization, allowing developers to ensure reliability and performance.
- **User-Facing Observability:** The project must turn the "black box" of AI into a transparent system for the end-user. This builds trust and provides users with a sense of control and understanding of the tool they are using.

Specification 9.2: Feature - The "System Health" Dashboard

Within its settings, LibreAssistant will feature a "System Health" or "Diagnostics" panel. This will be a user-friendly version of a developer's monitoring dashboard, providing transparent insights into the assistant's own operations.
This panel will provide:

- **Performance Metrics:** Simple, understandable visualizations of the assistant's average response time, the number of requests being handled, and the operational status of its connections to key external APIs (e.g., weather service, news service).
- **Resource Usage:** Clear information on how much local CPU, memory, and battery the assistant is consuming on the user's device, giving users control over its impact.
- **Error Log:** A human-readable log of any recent errors or failed requests (e.g., "Could not connect to the weather service at 8:05 AM"). This helps in user-led debugging and builds significant trust through radical transparency.

Specification 9.3: Temporal and Geospatial Intelligence

For LibreAssistant to be truly useful, it must have a sophisticated and privacy-preserving understanding of time and place.

- **Geospatial Awareness:** The assistant must be able to provide location-based services (e.g., "What's the nearest library?"). This will be achieved by integrating with open-source geospatial tools like OpenStreetMap. Crucially, all location data is considered highly sensitive personal data and must be handled according to the principles laid out in Part VI, stored only in the "Personal Data Vault" and subject to explicit, granular user control.
- **Temporal Intelligence:** The assistant's architecture must include robust components for parsing natural language requests related to time and for managing time-based data and tasks, such as schedules, reminders, and historical events.

Part X: Mechanical Engineering (Development Operations)

Core Principle

This section translates the principles of Book X of *De architectura*, which is dedicated to mechanical engineering—the machines used for construction, hydraulics, and warfare. This book completes the portrait of the architect as a master of all technologies related to the built environment. The modern parallel to these construction machines is the **DevOps toolchain and the automated systems that constitute the software development lifecycle (SDLC)**. These are the "engines" that build, test, deploy, and maintain the software. The machinery used to build the assistant is as important as the assistant's own architecture.

Specification 10.1: Principle of Automated and Transparent Construction

The entire software development lifecycle for LibreAssistant must be automated and transparent. The project must embrace a robust DevOps culture to enable a global community of contributors to build, test, and deploy code efficiently, reliably, and at scale. Relying on manual processes is the modern equivalent of a crane dropping a critical stone—it is slow, error-prone, and leads to failure.

- **Open DevOps Toolchain:** The project must be built using an entirely open-source DevOps toolchain. This includes using GitHub or GitLab for version control and collaboration, Jenkins or GitLab CI for CI/CD, and Docker for containerization.
- **Infrastructure as Code (IaC):** All infrastructure required to run the project's public services (e.g., community forum, documentation website, build servers) must be defined as code using an open-source tool like Terraform. This code will be stored in a public repository, ensuring the project's infrastructure is transparent, reproducible, auditable, and can be managed by the community. This is the ultimate form of automation, a machine that can build the factory itself.

Specification 10.2: Feature - The "One-Click" Contributor Environment

To lower the barrier to entry for new contributors and maximize community engagement, the project must provide a fully automated way to set up a complete local development

environment.
This will be implemented as a script or a set of Docker Compose files that, with a single command, will spin up a complete, containerized, local version of LibreAssistant and all its dependencies. This "machine" makes it incredibly easy for a new developer to go from cloning the repository to running the code and submitting their first contribution, embodying the principle of mechanical efficiency.

Specification 10.3: Fully Automated CI/CD Pipeline

The project's core repository must have a transparent, robust, and fully automated Continuous Integration and Continuous Deployment (CI/CD) pipeline. This automated "assembly line" ensures that all contributions are held to a high standard of quality and that the project can evolve rapidly and reliably, powered by its community.
The workflow will be as follows:

1. **Pull Request:** When a contributor submits a pull request, the CI system automatically triggers a build and runs a comprehensive suite of automated tests (e.g., unit tests, integration tests, static analysis, security scans).
2. **Peer Review:** Only if all automated checks pass is the code made available for peer review by core maintainers.
3. **Automated Merge and Deploy:** Once approved by maintainers, the code is automatically merged into the main branch. This triggers a new pipeline that builds, containerizes, and deploys the new version of the software to a staging environment for final validation.
4. **Release:** Official releases are then promoted from staging to production, with release notes automatically generated from commit messages.

Part XI: Project Licensing

Core Principle

This section formally declares the project's license, ensuring it aligns with the FOSS philosophy articulated in Part I and provides essential legal clarity for all contributors and users. The choice of license is a foundational architectural decision.

Specification 11.1: Official License Declaration

The LibreAssistant project, including all source code and associated documentation files created for the project, is licensed under the **MIT License** (SPDX identifier: MIT).
The MIT License is chosen for its simplicity and permissive nature. It is highly compatible with other licenses and places very few restrictions on reuse, allowing for use within proprietary software, provided the license and copyright notice are included. This broad permissiveness is essential for encouraging the widest possible adoption and contribution, which is a key goal of the FOSS philosophy and is necessary for the project to achieve its aims.

Specification 11.2: Full License Text

A file named LICENSE or LICENSE.txt containing the full, unmodified text of the MIT License

must be present in the root directory of every project repository. The canonical text is as follows :

```
Copyright (c) <year> The LibreAssistant Project Contributors

Permission is hereby granted, free of charge, to any person obtaining
a copy
of this software and associated documentation files (the "Software"),
to deal
in the Software without restriction, including without limitation the
rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or
sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be
included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT
SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE,
ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS
IN THE
SOFTWARE.
```

Specification 11.3: Implementation Guidelines

- **Copyright Holder:** The copyright holder shall be identified as "The LibreAssistant Project Contributors" to reflect the collective ownership of the project. The <year> placeholder should be updated to reflect the year of creation and subsequent major releases.
- **Package Metadata:** All software packages created for the project (e.g., in pyproject.toml, package.json files) must correctly identify the license as "MIT" to ensure it is properly indexed by package managers and software composition analysis tools.
- **Source File Headers:** While not strictly required by the MIT license itself, it is established as a project best practice that all source code files should contain a header comment indicating the license and copyright holder to ensure maximum clarity for downstream users and developers.

Part XII: Long-Term Research & Development

Roadmap

Core Principle

This final section outlines the project's future direction, translating the "Synthesis & Future Work" conclusions from the foundational paper into a concrete research and development roadmap. This roadmap ensures that LibreAssistant not only adheres to its principles at launch but continues to pioneer ethical, transparent, and user-centric AI. By pursuing these paths, LibreAssistant can move beyond being merely a piece of software and become a true architectural project: a durable, useful, and beautiful structure that provides both a private sanctuary for its user and a healthy, constructive gateway to the digital commons.

Roadmap Item 12.1: Develop a Framework for "Constitutional AI" in FOSS

- **Goal:** To move beyond simple content filters and build the project's ethical principles, as defined in the "Architect's Oath," directly into the AI's core architecture. The aim is to create an AI whose core programming reflects a constitutional commitment to healthy discourse.
- **Research Areas:** This will involve dedicated research and development into algorithmic approaches that can:
    - Actively promote viewpoint diversity in search results and summaries.
    - Quantify and transparently display indicators of source reliability and bias.
    - Detect and mitigate manipulative language patterns in source material.

Roadmap Item 12.2: Pioneer User-Centric, Privacy-Preserving Personalization

- **Goal:** To establish LibreAssistant as a leading research platform and reference implementation for advanced, privacy-preserving AI techniques. The objective is to create an assistant that can be highly personalized and useful without requiring the user to surrender their private data to a central server, providing a fundamental architectural alternative to dominant surveillance-based commercial AI models.
- **Research Areas:** This includes investing heavily in the practical implementation, optimization, and usability of:
    - Federated learning on heterogeneous, low-power devices.
    - Differential privacy with demonstrable utility for real-world features.
    - On-device model execution and fine-tuning.

Roadmap Item 12.3: Build a Model for a "Digital Guild"

- **Goal:** To perfect the social and governance architecture that surrounds the software, recognizing that the health of the community is essential for the long-term *firmitas* of the project itself.
- **Research Areas:** This involves explicitly researching and developing a sophisticated governance and contribution model based on the "digital guild" concept. Key initiatives

include:

- Creating clear and rewarding contribution pathways, from a first-time bug fix ("apprenticeship") to core maintenance ("mastery").
- Developing robust, open-source tools for collaborative design, asynchronous ethical review by the established guilds, and transparent, community-led decision-making.

Works cited

1. What is Free Software? - GNU Project, https://www.gnu.org/philosophy/free-sw.html 2. Free Software - FSFE, https://fsfe.org/freesoftware/ 3. The Free Software Definition - Wikipedia, https://en.wikipedia.org/wiki/The_Free_Software_Definition 4. Free and Open Source Software FOSS: Core Principles, https://osssoftware.org/blog/free-and-open-source-software-foss-core-principles/ 5. ACM FAccT 2025 Accepted Papers, https://facctconference.org/2025/acceptedpapers 6. Framework for FAccT check of AI legislation. | Download Scientific Diagram - ResearchGate, https://www.researchgate.net/figure/Framework-for-FAccT-check-of-AI-legislation_fig1_361433354 7. 7 Key Principles for GDPR Compliance in Software Development - Codific, https://codific.com/gdpr-compliance-software-development/ 8. Art. 5 GDPR – Principles relating to processing of personal data, https://gdpr-info.eu/art-5-gdpr/ 9. GDPR Developer's Guide to Achieving Compliance with Your CIAM - FusionAuth, https://fusionauth.io/articles/ciam/developers-guide-to-gdpr 10. What are the FAIR Data Principles? - Countway LibAnswers, https://asklib.hms.harvard.edu/nih-dmsp/faq/377100 11. FAIR data - Wikipedia, https://en.wikipedia.org/wiki/FAIR_data 12. FAIR Principles, https://www.go-fair.org/fair-principles/ 13. GDPR Compliance for Software in 15 Easy Steps - MindK.com, https://www.mindk.com/blog/how-to-make-your-software-gdpr-compliant/ 14. MIT License., https://www.mit.edu/~amini/LICENSE.md 15. MIT License - Wikipedia, https://en.wikipedia.org/wiki/MIT_License 16. MIT License, https://choosealicense.com/licenses/mit/